

Model Documentation

Project: Automated Algorithm Selection for 3D Nesting in Additive Manufacturing
Author: Jatin Rajani
Version: 3.0 (Final Optimised)
License: MIT

Table of Contents

- 1. [Problem Formulation](#)
- 2. [Algorithm Candidate Space](#)
- 3. [Model Architecture](#)
- 4. [Training Strategies](#)
- 5. [Training Strategy 1: Baseline \(From Scratch\)](#)
- 6. [Training Strategy 2: Strategist Meta-Model](#)
- 7. [Training Strategy 3: Final Optimised Model](#)
- 8. [Inference Pipeline](#)
- 9. [Performance Metrics](#)
- 10. [Model Artefacts](#)
- 11. [Deployment Considerations](#)
- 12. [Known Limitations and Future Work](#)

Problem Formulation

The core research problem is defined as follows:

Given a set of three-dimensional parts characterised by their geometric properties and a set of operator-supplied optimisation priorities, **predict** the single packing algorithm that will achieve the highest packing efficiency within the available build volume.

This is formally a **supervised multi-class classification** problem:

- **Input:** A feature vector x of dimension 23, derived from the geometric statistics of a part collection and three user-defined priority weights.
- **Output:** A predicted class label y from the set {blf_gravity, first_fit_decreasing, simulated_annealing, nfp_based, genetic, layer_decomposer}.
- **Training Signal:** Ground-truth labels obtained by exhaustive simulation of all six algorithms on each training scenario.

The class prior probabilities are unequal (see class distribution in dataset_documentation.md). No class reweighting or oversampling is applied; the model is trained to reflect the true distribution of optimal algorithm occurrences.

Algorithm Candidate Space

Six packing algorithms constitute the prediction target space. Their characteristics are summarised below to contextualise the model's decision boundaries.

Algorithm Key	Full Name	Dominant Advantage	Typical Optimal Scenario
nfp_based	No-Fit Polygon	Geometry-exact boundary computation for non-convex parts	Jobs dominated by flat plates, irregular polygonal parts
layer_decomposer	Layer Decomposer	Efficient horizontal layer stratification	Mixed-height assemblies with clear vertical structure

<code>genetic</code>	Genetic Algorithm	Global optimisation across vast placement search spaces	High-complexity scenes requiring density maximisation
<code>blf_gravity</code>	Bottom-Left with Gravity	Physically stable, low centre-of-mass placement	Jobs requiring gravitational stability
<code>first_fit_decreasing</code>	First Fit Decreasing	Extremely fast execution; sub-second for large jobs	Time-critical operations or prototyping pipelines
<code>simulated_annealing</code>	Simulated Annealing	Escaping local optima in constrained placements	Tightly constrained bounding volumes with irregular parts

Model Architecture

All three models in this research are instances of the **XGBoost Gradient Boosted Decision Tree** classifier, implemented via the `xgboost` Python library. This architecture was selected for the following reasons:

- Interpretability:** Individual tree paths can be inspected to understand which geometric features drive each prediction. Feature importance scores are directly available.
- Efficiency:** XGBoost inference is near-instantaneous (sub-millisecond) for a 23-dimensional input vector, making it suitable for real-time API responses.
- Performance on tabular data:** Gradient boosted trees consistently outperform deep neural networks on structured tabular data of this scale.
- Small memory footprint:** Even the largest model (`xgb_model_final.json` at approximately 138 MB) loads fully into memory and serves predictions with negligible latency.

Preprocessing Pipeline

Before features are passed to the model, the following preprocessing steps are applied:

- Missing value imputation:** Feature positions not computed from the input (e.g., packing density statistics when the simulation is bypassed in the API) are filled with zero.
- Standard scaling:** All features are normalised using a `StandardScaler` fitted on the training data. The scaler is serialised as `scaler_final.joblib`. This step is mandatory; passing raw unscaled features will produce degraded or invalid predictions.
- Feature ordering:** The 23 features must be provided in the exact order specified by `feature_names.json`. The `engine.py` module enforces this ordering programmatically.

Prediction Output

The model outputs a probability distribution across all six classes via `predict_proba`. The class with the highest probability is selected as the primary recommendation. The remaining classes are returned as ranked alternatives, sorted by their predicted probability in descending order.

Training Strategies

Three sequential training strategies were employed. Each strategy builds upon the previous, using insights from earlier experiments to improve classification performance. The progression demonstrates an increase in accuracy from 37% to 67%.

Training Strategy 1: Baseline (From Scratch)

Objective

Establish a performance baseline by training an XGBoost classifier directly on the full 24-column feature matrix using default hyperparameters.

Configuration

Parameter	Value
Estimator	XGBClassifier
Number of Estimators	100
Learning Rate	Default (0.3)
Max Depth	Default (6)
Feature Set	24-column (features.csv)
Validation	5-fold stratified cross-validation
Class Weighting	None

Outcome

Metric	Value
Test Accuracy	37%
Macro F1 Score	0.30
Macro Precision	0.33
Macro Recall	0.38

Analysis

The 37% baseline, while substantially above the 16.7% random chance threshold, reveals that the raw geometric features without priority weights do not provide sufficient signal to distinguish between many of the algorithm classes. In particular, simulated_annealing and first_fit_decreasing classes are poorly recalled due to their low representation in the training set and their geometric overlap with other classes.

The trained artefacts for this strategy are stored in:

- Packing_Algorithm_Project/Algorithm_Selection_for_3d_Packing_in_Additive_Manufacturing/model/xgb_model.j
- Packing_Algorithm_Project/Algorithm_Selection_for_3d_Packing_in_Additive_Manufacturing/model/scaler.jobl
- Packing_Algorithm_Project/Algorithm_Selection_for_3d_Packing_in_Additive_Manufacturing/model/label_encod

Training Strategy 2: Strategist Meta-Model

Objective

Improve classification performance by introducing a secondary "strategist" model that learns to correct the systematic errors of the baseline model. This is a form of stacked generalisation (model stacking).

Methodology

A second XGBoost classifier, referred to as the Strategist, was trained on a feature set augmented with the predicted probability distribution output of the Strategy 1 model. The Strategist thus has access to both the original geometric features and the baseline model's uncertainty signal, enabling it to learn corrective decision rules.

This approach leverages the observation that the baseline model's softmax output contains structured information about the difficulty of each scenario, even when the argmax prediction is incorrect.

Configuration

Parameter	Value
-----------	-------

Estimator	XGBClassifier
Input Features	24 original features + 6 predicted class probabilities from Strategy 1
Number of Estimators	100
Validation	5-fold stratified cross-validation

Outcome

Metric	Value
Test Accuracy	55%
Macro F1 Score	0.52
Macro Precision	0.52
Macro Recall	0.55

Analysis

The Strategist model achieves an 18-percentage-point improvement over the baseline. The improvement is largest on the majority classes (`nfp_based` , `layer_decomposer` , `genetic`) where the baseline model already had reasonable recall, and the Strategist could exploit structured error patterns.

The trained artefacts for this strategy are stored in:

- `Packing_Algorithm_Project/Algorithm_Selection_for_3d_Packing_in_Additive_Manufacturing_v2/model/model_st`
- `Packing_Algorithm_Project/Algorithm_Selection_for_3d_Packing_in_Additive_Manufacturing_v2/model/scaler_s`
- `Packing_Algorithm_Project/Algorithm_Selection_for_3d_Packing_in_Additive_Manufacturing_v2/model/label_en`

Training Strategy 3: Final Optimised Model

Objective

Maximise classification accuracy through systematic hyperparameter tuning, feature refinement, and training procedure improvements. This model is designated as the production model.

Key Changes from Strategy 2

- Feature Set Refinement:** The three convexity features (`convexity_mean` , `convexity_std` , `convexity_cv`) were removed and replaced with three user-supplied priority weights (`w_eff` , `w_stab` , `w_time`). This change serves two purposes: it introduces operator intent directly into the model input, and it removes features that were found to contribute marginally to information gain in the Strategy 1 analysis.
- Hyperparameter Optimisation:** A grid search was conducted over the following XGBoost hyperparameter space:
 - `n_estimators` : [100, 200, 300]
 - `max_depth` : [4, 6, 8]
 - `learning_rate` : [0.05, 0.1, 0.2]
 - `subsample` : [0.8, 1.0]
 - `colsample_bytree` : [0.8, 1.0]
 - `min_child_weight` : [1, 3, 5]
- Early Stopping:** Training was conducted with early stopping applied on a held-out validation set (15% of training data) to prevent overfitting. Training halted when the validation log-loss did not improve for 20 consecutive rounds.
- L1 and L2 Regularisation:** `reg_alpha` (L1) and `reg_lambda` (L2) regularisation terms were tuned to penalise model complexity and improve generalisation.

Configuration

Parameter	Tuned Value
Estimator	XGBClassifier
Input Features	23 columns (features_final_23cols.csv)
Number of Estimators	Determined by early stopping
Max Depth	Best from grid search
Learning Rate	Best from grid search
Early Stopping Rounds	20
Validation Fraction	15%
Validation	5-fold stratified cross-validation

Outcome

Metric	Value
Test Accuracy	67%
Macro F1 Score	0.65
Macro Precision	0.64
Macro Recall	0.70

The trained artefacts for this strategy are stored in:

- Packing_Algorithm_Project/Algorithm_Selection_for_3d_Packing_in_Additive_Manufacturing_v2/model2/xgb_mod
- Packing_Algorithm_Project/Algorithm_Selection_for_3d_Packing_in_Additive_Manufacturing_v2/model2/scaler_
- Packing_Algorithm_Project/Algorithm_Selection_for_3d_Packing_in_Additive_Manufacturing_v2/model2/label_e
- Packing_Algorithm_Project/Algorithm_Selection_for_3d_Packing_in_Additive_Manufacturing_v2/model2/feature

Production copies of the three model artefacts are also present in:

- research_backend/prediction/ml_model/

Inference Pipeline

The production inference pipeline is implemented in research_backend/prediction/engine.py . The following steps are executed for every prediction request.

Step 1: Artifact Loading (Lazy, Singleton)

The ensure_artifacts_loaded() function loads the three model artefacts (XGBoost model, StandardScaler, and LabelEncoder) into memory the first time a prediction is requested, and thereafter retains them as module-level singletons. Subsequent requests are served without disk I/O.

```
# Pseudocode representation of the loading logic
if model is None or scaler is None or label_encoder is None:
    model = XGBClassifier(); model.load_model("xgb_model_final.json")
    scaler = joblib.load("scaler_final.joblib")
    label_encoder = joblib.load("label_encoder_final.joblib")
```

Step 2: Geometry Feature Extraction

The `generate_geometry_features(counts, priorities)` function accepts the user-supplied shape counts and priority weights and constructs the 23-dimensional input vector.

For each geometry type (cubes, plates, spheres, cylinders, cones), `trimesh.creation` is used to instantiate mesh objects at the standard dimensions. Volume statistics (`total_volume`, `volume_mean`, `volume_std`, `volume_cv`) and object count (`n_objects`) are computed directly from these meshes.

The remaining features that require full simulation (`packing_density_*`, `sphericity_*`, `aspect_ratio_*`, `face_count_*`, `com_height_*`) are defaulted to zero for API requests, as running the full physics simulation per request would be computationally prohibitive. The three priority weights are inserted directly.

Step 3: Scaling

The feature vector is passed to `scaler.transform()`. This applies the same mean-centering and standard deviation normalisation that was applied to the training data.

Step 4: Prediction

The scaled feature vector is passed to `model.predict_proba()`, which returns a probability distribution across all six algorithm classes. The class corresponding to the highest probability is selected as the primary recommendation.

Step 5: Response Construction

The response dictionary includes:

- `algorithm`: The string name of the recommended algorithm.
- `confidence`: The predicted probability of the recommended class, expressed as a float in [0, 1].
- `alternatives`: A list of up to three alternative algorithms, each with their predicted probability score, sorted in descending order of probability.
- `explanation`: A pre-written contextual explanation string generated in `views.py` based on the predicted algorithm name.

Performance Metrics

Progressive Performance Summary

Strategy	Accuracy	F1 (Macro)	Precision (Macro)	Recall (Macro)
Strategy 1 (Baseline)	37%	0.30	0.33	0.38
Strategy 2 (Strategist)	55%	0.52	0.52	0.55
Strategy 3 (Final)	67%	0.65	0.64	0.70

Comparison to Baselines

Comparison	Result
Random chance (6 classes, uniform)	~16.7%
Always-predict-most-frequent (<code>nfp_based</code>)	~31.0%
Strategy 3 vs. random	4.0x improvement
Strategy 3 vs. most-frequent heuristic	2.16x improvement

Validation Protocol

All reported metrics are computed on a held-out test set that was excluded from all training and hyperparameter tuning. The primary validation methodology is 5-fold stratified cross-validation, which ensures that the reported metrics are not a function of a single favourable train-test split.

Model Artefacts

File Summary

File	Purpose	Format
xgb_model_final.json	XGBoost model weights (Strategy 3)	XGBoost native JSON
scaler_final.joblib	StandardScaler fitted on training features	joblib binary
label_encoder_final.joblib	LabelEncoder mapping integer indices to algorithm names	joblib binary
feature_names.json	Ordered list of the 23 input feature names	JSON
xgb_model.json	XGBoost model weights (Strategy 1)	XGBoost native JSON
scaler.joblib	StandardScaler for Strategy 1	joblib binary
label_encoder.joblib	LabelEncoder for Strategy 1	joblib binary
model_strategist.json	XGBoost Strategist model weights (Strategy 2)	XGBoost native JSON
scaler_strategist.joblib	StandardScaler for Strategy 2	joblib binary
label_encoder_strategist.joblib	LabelEncoder for Strategy 2	joblib binary

Loading Model Artefacts Programmatically

```

import joblib
import xgboost as xgb

# Load the production model
model = xgb.XGBClassifier()
model.load_model("path/to/xgb_model_final.json")

scaler = joblib.load("path/to/scaler_final.joblib")
label_encoder = joblib.load("path/to/label_encoder_final.joblib")

# Prepare a 23-dimensional feature vector (must be ordered per feature_names.json)
import pandas as pd
import json

with open("path/to/feature_names.json") as f:
    feature_names = json.load(f)

# Construct feature dictionary; replace with actual computed values
feature_dict = {name: 0.0 for name in feature_names}
feature_dict["n_objects"] = 10
feature_dict["total_volume"] = 5000.0
feature_dict["w_eff"] = 0.8
feature_dict["w_stab"] = 0.6
feature_dict["w_time"] = 0.4

input_df = pd.DataFrame([feature_dict])

# Scale and predict
input_scaled = scaler.transform(input_df)
predicted_probabilities = model.predict_proba(input_scaled)[0]
predicted_index = predicted_probabilities.argmax()
predicted_algorithm = label_encoder.inverse_transform([predicted_index])[0]

print(f"Recommended Algorithm: {predicted_algorithm}")
print(f"Confidence: {predicted_probabilities[predicted_index]:.3f}")

```

Deployment Considerations

Production Environment

The model is deployed as part of the Django REST API defined in `research_backend/`. The following recommendations apply to production deployments:

- **Server:** Use Gunicorn with at least 2 worker processes. The `WEB_CONCURRENCY` environment variable controls the worker count.
- **Memory:** Each worker process loads all three model artefacts into memory. The total footprint per worker is approximately 160-200 MB. Provision at least 512 MB of RAM per worker.
- **HTTPS:** The `settings.py` file configures `SECURE_SSL_REDIRECT`, `SECURE_HSTS_SECONDS`, and `SESSION_COOKIE_SECURE` for production. Ensure that the deployment infrastructure terminates TLS upstream and passes the `X-Forwarded-Proto: https` header.
- **CORS:** Update `DJANGO_CORS_ALLOWED_ORIGINS` and `DJANGO_CSRF_TRUSTED_ORIGINS` in the production `.env` file to match the deployed frontend domain.
- **Secret Key:** Generate a cryptographically random `DJANGO_SECRET_KEY` value for each deployment environment. Never reuse the development placeholder.

Scalability

For high-throughput deployments, the model artefacts may be loaded once at server startup using Django's `AppConfig.ready()` hook and stored in a process-global cache. Alternatively, a dedicated inference service (e.g., TensorFlow Serving, Seldon, or a custom FastAPI microservice) may be more appropriate at scale.

Known Limitations and Future Work

Current Limitations

1. **Geometry simplification in the API:** The production API approximates geometry features by instantiating canonical primitive meshes rather than accepting actual part STL files. This means that jobs containing custom CAD geometries may not be characterised with the same accuracy as jobs composed of the five training geometry types.
2. **Class imbalance:** The minority classes (`simulated_annealing` at 4.9% and `first_fit_decreasing` at 7.3%) are likely to be under-predicted relative to their true optimal frequency for scenarios outside the training distribution.
3. **Single build volume:** The model was trained exclusively on a 300 x 300 x 400 mm build volume. Its generalisation to different printer sizes has not been evaluated.
4. **No support structure modelling:** The dataset and model do not account for the support structure overhead required for overhanging geometries in FDM or SLA printing. Including support volume as a feature would improve the practical utility of the recommendations.

Future Research Directions

1. **STL input pipeline:** Develop a feature extraction pipeline that accepts actual STL files, computes the full 23-feature vector from real mesh geometry, and removes the requirement for canonical primitive approximation.
2. **Dataset expansion with real printer data:** Augment the synthetic dataset with labelled scenarios derived from real production nesting jobs to reduce the domain gap.
3. **Deep learning comparison:** Evaluate Graph Neural Network or point cloud classification approaches against the XGBoost baseline to determine whether geometric deep learning offers a material advantage.
4. **Multi-objective output:** Extend the model to output a Pareto-optimal set of algorithm recommendations when the user's priority weights indicate conflicting objectives.
5. **Online learning:** Investigate whether the model can be updated incrementally as new scenarios and their outcomes are observed in a production deployment.